

Pokémon-Inspired Portfolio

Engineering Blueprint + Coding-Agent Instructions

Version 1.0 | Prepared for Uttkarsh | Status: DRAFT FOR REVIEW - no implementation has started

Contents

How to read this document	3
PART I - ENGINEERING BLUEPRINT	4
0. Document Reconciliation	5
1. Executive Engineering Summary	6
2. Architecture Principles	6
3. System Architecture	7
4. React / Phaser Boundary	7
5. State Architecture	8
6. Battle State Machine	9
7. Question Tree Architecture	10
8. Project / Audience / Content Architecture	11
9. Audience Architecture	12
10. Overworld Architecture	12
11. UI / Menu Architecture	13
12. Rendering + Animation Architecture	13
13. Input Architecture	13
14. Routing + SEO Architecture	14
15. Asset + Audio Architecture	14
16. Persistence Architecture	15
17. Performance Architecture	15

18. Accessibility Architecture	15
19. Testing Architecture	15
20. Error Handling / Resilience	16
21. Security / Browser Concerns	16
22. Deployment Architecture	16
23. Target Repository Structure	17
24. Architecture Decision Records	17
25. Phased Implementation Plan	18
26. Parallel Work Plan	18
27. Technical Risks + Mitigations	18
28. Pre-Implementation Checklist	19

PART II - CODING-AGENT INSTRUCTIONS **20**

A. How to run the agents	21
B. Master Brief (paste at the start of EVERY agent session)	21
C. Phase prompts (paste ONE at a time, after the Master Brief)	22
D. Phase Report template (agents must end every phase with this)	26
E. Escalation triggers (agent must stop and ask)	26
F. Owner review checklist (per phase)	26
G. Pre-flight before Phase 0	27

How to read this document

Part I is the architecture and phased plan (what to build and why). **Part II** is the exact instruction set for coding agents: one Master Brief that goes into every agent session, plus one copy-paste prompt per phase. Nothing here is implementation code.

- Recommended decisions D1-D10 (Section 0.3) are assumed by Part II. If you flip any, edit the "Locked Decisions" block in the Master Brief before handing it over.
- Hand agents **one phase at a time**. Review the phase report before releasing the next.

PART I - ENGINEERING BLUEPRINT

0. Document Reconciliation

0.1 Precedence rule used

1. Engineering brief (most recent, explicit).
2. PRD v1.0 and Functional Interaction/State v1.0 (final baseline).
3. Other v1.0 documents within their own domain.
4. UI/UX v0.1 for visual tokens only (self-described draft; predates several locked decisions).

0.2 Contradictions and gaps

ID	Conflict or gap	Resolution	Sign-off
C1	UI/UX says challenger stays visible in battle; PRD and brief say challenger appears only during dialogue.	PRD and brief win.	No
C2	UI/UX makes Uttkarsh the lower-field "trainer"; Functional and brief make the visitor the player-side character and Uttkarsh the opponent.	Functional wins. Visitor sprite lower-left; Uttkarsh + project logo upper-right.	No (art note)
C3	UI/UX requires YES/NO link confirmation; PRD, Functional, brief say LINK opens immediately.	PRD wins. LinkDialog removed.	No
C4	UI/UX says 2-3 NPCs; PRD/World say exactly 2 ordinary + 1 challenger.	PRD wins: 3 NPC entities.	No
C5	Depth numbering: Content example gives four answer levels; PRD/Functional give Level 0 = summary root, answers at depths 1-3.	Root is a container; answers at depths 1-3. MAX_ANSWER_DEPTH = 3 (one constant).	Yes (D1)
C6	Brief wants "nothing deeper here" at max depth; three docs want auto-return to parent after a leaf answer (so DETAILS can never be pressed at a leaf).	Reducer supports both; default is auto-return.	Yes (D3)
C7	"Same depth" on project switch is ill-defined at depth 2+ (different parents).	Shared topicKey; resolve path by key; fall back to nearest valid ancestor.	Yes (D2)
C8	Party selection "full" transition vs switching "shortened".	Opening PARTY = full; confirming a project = short switch effect.	No
C9	Audience labels differ ("Software Engineer" vs "I'm an engineer").	Labels are data; IDs are recruiter / engineer / friend.	No
C10	Battle background reuses encounter environment vs "original battle environment" in UI/Asset specs.	Paused Phaser world under DOM battle UI is the backdrop. No new asset.	No
C11	Player Menu EXIT destination undefined.	Menu EXIT goes to /about.	Yes (D5)
C12	X/Y availability in battle/cutscenes unspecified.	Disabled during ENCOUNTER, VS, BATTLE, TRANSITION.	Yes (D4)
C13	Approach 6-10 s (brief) vs conversation 6-10 s (FTE).	One budget: detection through pre-VS line, excluding reading time.	No
C14	Six-project selection not final; UI/UX lists six.	UI/UX six are seed content; engine treats all as data.	No
C15	~32x32 sprites vs 16x16 collision on a 15x10-tile screen.	Collision = one tile at feet; sprite anchored bottom-center. Verify in art phase.	Art check
C16	"Persists for the current session" ambiguous on refresh.	Session flags in sessionStorage; refresh restarts at spawn, skips narration.	Yes (D6)
C17	Brief mentions a "very small opening world message" not in FTE.	Assumed to be the narration.	Confirm
C18	Menu > Projects selection outcome undefined.	Read-only project panel with LINK and "open full page". Not a battle.	Yes (D10)
C19	Integer scaling makes a 240 px canvas tiny on portrait phones.	Device-pixel integer scaling; landscape recommended.	Yes (D7)
C20	Functional: B exits interiors; classic games use a door mat.	B plus an exit tile.	Confirm (D9)
G1	Audio blocked before user gesture.	Cue plays on PRESS START (unlocks audio).	No
G2	LOS geometry, walk speed, Resume USE target unspecified.	Assumptions in Sections 10 and 14.	D8

0.3 Decisions to close before Phase 1

#	Decision	Recommendation
D1	Depth model	3 answer levels under a summary root.
D2	Party-switch landing	Key-path resolution; land on topic list at same depth, cursor on matching topic; no auto-played answer.
D3	Leaf behavior	Hard auto-return. "Nothing deeper" kept as reducer guard.
D4	X/Y in encounter and battle	Disabled.
D5	Menu EXIT and Resume USE	EXIT to /about. USE Resume opens PDF in new tab; /resume is the HTML view.
D6	Session semantics	sessionStorage for progress. Refresh restarts at spawn, no narration.
D7	Mobile	Landscape-first, device-pixel integer scaling.
D8	LOS	Straight line in challenger facing direction, up to 3 tiles, blocked by solids.
D9	Interior exit	B plus exit tile.
D10	Menu Projects selection	Read-only panel.

1. Executive Engineering Summary

The product is a deterministic, data-driven interview engine wearing a game skin. The overworld supplies atmosphere and discovery. Everything professional is data rendered twice: once in the pixel UI and once in normal HTML routes.

1. **Three layers.** Static typed content at the bottom; a pure TypeScript core with no Phaser/React/DOM; thin shells on top (Phaser for world pixels, React for all UI, Next.js for routes).
2. **React renders all in-game UI.** Dialogue, menus, battle, VS, hints are DOM inside a scaled 240x160 viewport over the Phaser canvas (ADR-02).
3. **The battle is a pure reducer over tree data.** The engine never names a project or an audience.
4. **Phaser is paused, not destroyed, during battle.** It becomes the backdrop; "exact position on return" falls out for free.
5. **Transitions live above both canvas and DOM** (ADR-04).
6. **No backend.** Typed TS content validated by Zod and graph checks in CI; session/local storage; static CDN hosting.
7. **Build order:** headless battle core, battle UI with no world, world, glued vertical slice (Title to exit), menus and web routes, full content and polish, hardening.

Stack verdict: no required deviations; three optional refinements (ADR-02, ADR-03, ADR-05).

2. Architecture Principles

1. **Content is data, logic is generic.** No engine code names a project, audience, or tree node.
2. **Pure core, impure shells.** Battle, encounter, sim, LOS are deterministic functions of state, events, and an injected clock.
3. **One owner per state variable.** No global store; no cross-store writes.
4. **Two renderers, one source.** In-game screens and web routes read the same view-model selectors.
5. **The game is never a gate.** Every essential fact has a route and a direct menu path.
6. **Fail soft.** Broken asset or node degrades to a placeholder or parent level, never a dead screen.
7. **Deterministic time.** No setTimeout in core or cutscenes; a Clock service is injected.
8. **Stable IDs over indexes.**
9. **Validate at build time first, runtime second.** Invalid content fails CI.
10. **No speculative engine.** About eight cutscene step types, one closed predicate set, no scripting language.

3. System Architecture

```
Next.js App Router (src/app)
/ (game, client-only) /projects /projects/[slug] /skills /experience /about /resume
  | mounts                                     | reads
runtime/ Orchestrator, Bridge, Stores, Services <---> content/ (typed TS + Zod)
  | intents                                   | commands/events      ^ types
ui/ React pixel UI   core/ pure TS: world sim,      domain/ types, schemas,
+ TransitionLayer    battle, encounter, flow,      tree utils
                    input
                    game/ Phaser world view (renders WorldSim snapshots)
```

Dependency rules (enforced by dependency-cruiser + ESLint in CI)

Layer	May import	Must not import
domain	nothing	everything else
content	domain	core, game, ui, runtime
core	domain	content data, Phaser, React, DOM
game	core, domain, runtime/bridge types	React, content
ui	core, domain, runtime hooks/selectors	Phaser
runtime	core, domain, content via provider	Phaser objects, React components
app	everything via public entry points	deep imports

Subsystems

Subsystem	Responsibility	Owner	Failure modes	Tests
WorldSim	Tile movement, collision, NPC routes, interactions, LOS	core/world	bad map data, NPC deadlock	unit, replay hash
Battle reducer	All battle navigation	core/battle	invalid transition, missing node	exhaustive table, property tests
EncounterMachine	First/repeat encounter choreography	core/encounter	interrupted sequence	unit, E2E
FlowMachine	Top-level mode (TITLE to BATTLE)	core/flow	illegal mode jump	transition-table test
InputRouter	Action abstraction, context stack	core/input	stuck keys, double activation	unit, E2E
Orchestrator	Wires machines to shells, runs effects	runtime	ack timeouts	integration
Phaser view	Renders sim, maps, sprites, backdrop	game	asset fail, WebGL loss	visual regression
Pixel UI	Dialogue, menus, battle, VS, hints	ui	focus loss, overflow	component, axe
TransitionLayer	Wipes, fades, VS timing	ui	stuck lock	timeout test
Content layer	Typed data, validation	content + scripts	bad tree	validators
AudioService	Music, SFX, unlock, mute	runtime	autoplay block, decode failure	mock
Persistence	Storage adapters, schema versions	runtime	corrupt JSON, disabled storage	unit

4. React / Phaser Boundary

Concern	Phaser	React	Core
World pixels, tilemap, sprites, walk animation	Yes		
Movement, collision, LOS, NPC routes			Yes (Phaser renders snapshots)
Dialogue, menus, battle UI, VS, hints, Options		Yes	
Battle navigation			Yes
Battle backdrop	Yes (paused world, locked camera)		
Transition wipes		Yes (top DOM layer)	
Text and portfolio content	never	Yes	

Concern	Phaser	React	Core
Audio			runtime service

GameBridge contract

- **Commands (runtime to Phaser):** loadMap(id), setCameraRoom, pauseWorld, resumeWorld, playEntityRoute(entityId, steps), faceEntity, showExclaim(entityId), snapshot(). Each returns a Promise, acked by the view, raced against a timeout.
- **Events (Phaser to runtime):** stepCompleted, interactionRequested(targetId), mapArrived, worldReady, assetFailed(key).
- **Hard rules:** Phaser receives only IDs, coordinates, animation keys - never prose. Phaser does not import content or React. React does not touch Phaser objects. Phaser never owns input; it reads held-direction state from InputRouter.

First-encounter sequence

```

Phaser/WorldSim    Orchestrator/Encounter    React UI    TransitionLayer
stepCompleted --->  LOS? yes -> DETECTED
                   lock input, Clock.wait(300)
<-- face+exclaim -- ANTICIPATION -> "!"
<-- route(approach) APPROACH
-- ack ----->    INTRO_DIALOGUE -----> dialogue x3
                   AUDIENCE_SELECT -----> choice -> audienceId
                   PRE_VS -----> "Show me what you have got"
                   VS -----> VS screen (~1.5 s)
                   pauseWorld -----> battle wipe
                   Battle reducer created -> BattleView mounts

```

5. State Architecture

Stores are small Zustand vanilla stores: settingsStore (local), progressStore (session), hintStore (local), uiStore (transient). The flow, battle, and encounter machines are reducer instances owned by the orchestrator, not stores.

State	Class	Owner	Lifetime / persistence
Player tile, facing, step progress, moving	Transient	WorldSim	live
Current map ID, camera room	Transient (room derived)	WorldSim	live
Map stack (exterior return point)	Transient	WorldSim	live
NPC tile, facing, route index, wait timer	Transient	WorldSim	live
Challenger state	Transient	WorldSim entity + EncounterMachine	live
Encounter anchor (map, tile, facing)	Transient	Orchestrator	battle duration
First-encounter completed	Persistent (session)	progressStore	sessionStorage
Intro seen	Persistent (session)	progressStore	sessionStorage
Current audience	Transient	BattleContext	per encounter
Current project	Transient	BattleContext	per battle
Party order	Derived	content + audience	n/a
Current tree	Derived	(project, audience) lookup	n/a
Focus node, answer phase, page index	Transient	BattleContext	per battle
Cursor index, typing progress	Transient, view-local	React component	unmount
Visited nodes	Persistent (session)	progressStore	sessionStorage
Reaction cooldown counter	Transient	BattleContext	per battle
Discovered eggs / collectibles	Persistent (session)	progressStore	sessionStorage; no UI counter
Tutorial hints seen	Persistent	hintStore	localStorage
Music, SFX, text speed	Persistent	settingsStore	localStorage
Reduced motion	Persistent + derived	settingsStore + matchMedia	localStorage
Menu stack and selection	Transient	FlowMachine + uiStore	live
Top-level mode	Transient	FlowMachine	live

State	Class	Owner	Lifetime / persistence
Transition in progress	Transient	TransitionDirector	live
Input context stack	Transient	InputRouter	live
Asset load status	Transient	AssetLoader	live
Route	URL	Next router	URL
Viewport scale, DPR	Derived	React hook	live
Static content	Static	content/	build

URL state: path only. Menu and battle state are not encoded in the URL (battle is not deep-linkable in MVP).

6. Battle State Machine

Context (serializable)

```
interface BattleContext {
  audienceId: AudienceId; projectId: ProjectId; partyOrder: ProjectId[];
  view: 'entry'|'sendout'|'root'|'topics'|'answer'|'party'
    |'switching'|'notice'|'reaction'|'exiting'|'closed';
  focusId: NodeId | null; // null = root
  answerPhase?: 'reading'|'commands';
  pageIndex: number;
  resume?: ViewSnapshot; // where PARTY/NOTICE/REACTION return to
  visited: ReadonlySet<string>;
  reactionCooldown: number;
}
```

Semantics

- A node's view is either **answer** (reading it) or **topics** (choosing among its children). ROOT is the summary view of focusId = null.
- **Back rule:** BACK from any view on node N goes to topics of parent(N). BACK from topics(null) goes to ROOT. BACK at ROOT exits (Functional Section 10).
- **Depth:** answer(N) is at depth |N|; topics(N) is at depth |N|+1.

Transitions

From	Event	To / effect
entry	TRANSITION_DONE	sendout
sendout	ADVANCE (last page)	reaction(project-entry) then root
root	DETAILS	topics(null); if no topics, notice
root	LINK	root + effect OPEN_LINK (no state change)
root	OPEN_PARTY	party (resume = root)
root	EXIT / BACK	exiting
topics(N)	SELECT_TOPIC(c in children(N))	answer(c, page 0, reading); reaction detail-open or known-content if cooldown allows
topics(N)	BACK	topics(parent(N)), or root if N is null
topics	OPEN_PARTY / EXIT	party / exiting
answer(reading)	ADVANCE (more pages)	next page
answer(reading)	ADVANCE (last page), N has children	answer(commands)
answer(reading)	ADVANCE (last page), N is leaf	topics(parent(N)); reaction answer-return
answer(reading)	BACK	topics(parent(N))
answer(commands)	DETAILS	topics(N)
answer(commands)	BACK	topics(parent(N))
answer(commands)	OPEN_PARTY / EXIT	party / exiting
answer, leaf	DETAILS	notice("nothing deeper here") then back (guard; unreachable under D3 default)
party	SELECT_PROJECT(other)	switching

From	Event	To / effect
party	SELECT_PROJECT(same) / BACK	resume view
switching	TRANSITION_DONE	short send-out line, reaction(project-switch), resolved view
reaction / notice	ADVANCE	resume view
exiting	TRANSITION_DONE	closed (emit BATTLE_ENDED)
any	ABORT	closed, no further effects

Events arriving during locked states (entry, switching, exiting) are dropped, not queued. Key-repeat for A/B is ignored.

Switch resolution

resolveSwitch(oldTree, oldView, newTree) returns a ViewSnapshot:

1. Take the old focus path as a list of topicKeys (root to focus).
2. Walk the new tree matching children by topicKey at each depth; stop at the first miss.
3. If old view was answer(P): land on topics(parent(P)), cursor on matching topic if it exists. If old view was topics(P): land on topics(P'). If the resolved path is empty: land on root.
4. No content is invented; missing branches fall back to nearest valid parent.

Edge cases

Case	Behavior
X/Y during battle	Ignored (D4).
EXIT while typing	EXIT not offered until commands visible. BACK during typing interrupts to parent.
Switch at depth 2	Key-path resolution, then reaction.
Browser Back	Mapped to B via history sentinel; at ROOT exits battle.
Refresh	Whole game restarts; battle not persisted (D6).
Route change / unmount	Orchestrator aborts timers, sends ABORT, destroys world.
Missing tree/node at runtime	Result-based resolver errors; reducer goes to neutral notice and returns to last valid view. Project hidden from PARTY and logged.
Logo/asset failure	Text badge placeholder with project name and Type.
LINK with invalid URL	Button disabled with neutral notice (validator should prevent).

7. Question Tree Architecture

Authored model

```

type ProjectId = Brand<string, 'ProjectId'>; type AudienceId = Brand<string, 'AudienceId'>;
type NodeId = Brand<string, 'NodeId'>;
interface AnswerBlock { pages: string[] } // authored pages, plain text
interface TopicNode {
  id: string; topicKey?: string; label: string;
  answer: AnswerBlock; // required
  children?: TopicNode[]; // 2-4 preferred
  reactionHook?: string;
}
interface QuestionTree {
  projectId: ProjectId; audienceId: AudienceId;
  status: 'stub' | 'authored';
  rootPrompt?: AnswerBlock; topics: TopicNode[]; // depth-1 nodes
}

```

Pages are authored (not auto-paginated), so pagination is deterministic and independent of font metrics. A validator caps characters per page (tunable constant until the font is chosen).

Compiled form

```
interface CompiledNode { id: NodeId; parent: NodeId|null; depth: 1|2|3; childIds: NodeId[];
    topicKey?: string; label: string; answer: AnswerBlock }
interface CompiledTree { projectId; audienceId; nodes: Record<NodeId,CompiledNode>; rootChildren: NodeId[] }
```

compileTree() runs at build time; the reducer only sees compiled trees. MAX_ANSWER_DEPTH = 3 lives in domain/limits.ts.

Engine surface (signatures only)

```
compileTree(tree): Result<CompiledTree, TreeError[]>
getChildren(tree, focus: NodeId|null): CompiledNode[]
battleReduce(ctx, ev, deps): { ctx; effects: BattleEffect[] }
resolveSwitch(...): ViewSnapshot
```

Validators (CI + dev watcher)

ID	Rule
V1	Every active project has a tree for every registered audience.
V2	Every node has a non-empty answer.
V3	Depth <= MAX_ANSWER_DEPTH.
V4	Node IDs unique per tree.
V5	Root has >= 2 topics; other menus have 2-4 children when children exist.
V6	topicKey unique among siblings.
V7	Each project has a Type that exists in the Type registry.
V8	Each project has exactly one primary link, valid https:, allowlist passes.
V9	Party has exactly 6 active projects and a valid default first.
V10	Page-length limits.
V11	No stub trees in a release build.
V12	No percentage/metric patterns unless flagged sourced: true (soft lint; human review still required).
V13	Every root topicKey matches the shared Level 1 vocabulary.
V14	Random event walks never reach an undefined reducer state.

8. Project / Audience / Content Architecture

```
interface ProjectDef {
  id; slug; name; tagline; type: string /*ProjectTypeDef.id*/;
  technologies: string[] /*skill IDs*/; role; period; team?;
  summary: AnswerBlock; overview: string[]; impact: string[]; // no invented metrics
  links: { primary: LinkDef; others: LinkDef[] };
  visual: { logo: AssetRef; thumb: AssetRef; alt: string };
}
interface PartyConfig { activeProjectIds: ProjectId[] /*exactly 6*/; defaultFirst: ProjectId;
  orderByAudience?: Partial<Record<AudienceId, ProjectId[]>> }
```

Shared Level 1 vocabulary: the four canonical topics (what, how, why, hard) are aligned topicKeys at depth 1. Deeper topicKeys are optional and improve switch continuity.

Other content modules: skills (no proficiency numbers), experience, trainer-card, bag, world (NPC dialogue variants, signs, objects, eggs, collectibles), hints, reactions, ui-strings.

Seed projects (placeholders): Acko Clinic (Backend), Buy Me A Coffee dApp (Blockchain), PDF Q&A (AI), Mini Port Scanner (Networking), NomNom Planner (Full-Stack/SaaS), IoT Weather Monitoring (IoT/Backend). Nothing in the documents describes project internals - every answer must come from you.

9. Audience Architecture

```
interface AudienceDef {
  id: AudienceId; choiceLabel: string; challengerTitle: string; announcement: string;
  portraitKey: string; greetings: { first?: string[]; repeat: string[] }; reactions: ReactionPool;
}
type ReactionContext = 'project-entry'|'project-switch'|'detail-open'|'answer-return'|'known-content';
```

- Audience-specific behavior comes from data: trees, reactions, greetings, portrait, Party order.
- Engine code never branches on an audience ID; a lint check greps for the three ID literals outside content/ and tests.
- Reaction selection is deterministic: round-robin per context with a minimum cooldown. Entry and switch reactions bypass the cooldown.
- Audience is a lens, not a permission boundary; reviewed as a content item.

10. Overworld Architecture

- **Maps:** Tiled JSON compiled by a compile-maps script into normalized MapData shared by sim and Phaser (single collision source). 16x16 tiles; screen 15x10 tiles; exterior is a multiple of screen size. Layers: ground, decor, collision, above, objects. Object types: spawn, door, sign, object, npc, challenger, collectible, room. MVP maps: town + interior-1..3.
- **Simulation:** fixed-step ticks; entities occupy tiles; one step = one tile with tunable duration (from ~100 ms walk-frame target; needs playtesting). Collision flags: SOLID, WATER (blocked), GRASS (cosmetic only, no encounter mechanic).
- **Player controller:** samples held direction at step boundaries; facing updates on input; step only if target walkable.
- **NPC controller:** scripted routes (loop or pingpong, with waits); reserves destination tile at step start; waits if blocked; on interaction turns to face Uttkarsh and resumes after dialogue.
- **Interactions:** Enter interacts with the tile in front. **Doors:** tiny fade; push exterior (map, tile, facing) on the map stack; load interior. **Interiors:** fixed one-screen; B or exit tile leaves (D9).
- **Camera:** instant room snap; room = floor(tile / (15,10)) or authored rect. Opening reveal = short scripted sequence of room snaps ending at spawn room.
- **Eggs/collectibles:** interactable objects with effect data and optional unlocks; discovery writes ID to progressStore; no UI counter.
- **NPC dialogue variance:** ordered DialogueVariant { when?: Predicate; lines }, first match wins. Predicate is a closed set of flag checks over { firstEncounterDone, discovered(id) }.

Line of sight and challenger trigger (D8)

- Straight line from the challenger in its facing direction, 1-3 tiles, blocked by SOLID tiles and entities.
- Evaluated only at tile-step completion and on map arrival. Active only while firstEncounterDone is false and mode is OVERWORLD.
- Later encounters: Enter on the challenger; LOS disabled.
- Placement validator: LOS lane tiles must be walkable, reachable from spawn, and cross a normal route.

Encounter choreography

EncounterMachine is a step-list runner with about eight step types: wait, face, exclaim, move, say, choice, vs, startBattle. Steps are data. Locked copy comes from content. Repeat encounter (assumption): audience greeting, audience choice, "Show me what you have got", VS screen (open question whether VS repeats).

Transitions

TransitionSpec { id, durationMs, reducedVariant }. Types: door-fade, battle-wipe, vs, switch-short, exit-short. Owned by TransitionDirector, rendered by the DOM TransitionLayer.

11. UI / Menu Architecture

- **Screen stack:** menus are MenuDef data; ScreenStack in uiStore; B pops. Menus read view-model selectors, never raw content.
- **Components:** GameViewport, Window, MenuList, CommandGrid, DialogBox, Cursor, BagScreen, PokedexList/Detail, PartyScreen, BattleScreen, DetailsTree, TrainerCard, ExperienceScreen, OptionsScreen, TouchController, VsScreen, HintToast, TransitionLayer. (No LinkDialog.)
- **Dialogue:** DialogueService takes a queue of requests and returns a Promise of the result. Typewriter driven by the clock at settings speed; Enter reveals full text, then advances.
- **Battle UI:** pure function of battle context; Phaser shows through as backdrop.
- **VS screen:** DOM + CSS, ~1.5 s.
- **Hints:** HintDef { id, trigger, text, priority }; HintService subscribes to domain events, shows one at a time, stores seen IDs; Options can reset.
- **Menu > Projects (D10):** read-only panel with LINK and "open full page".
- **Pokédex:** list-first with category grouping, optional search, no proficiency bars.

12. Rendering + Animation Architecture

Layer stack (bottom to top) inside GameViewport (240x160 logical): 1 Phaser canvas; 2 DOM pixel UI; 3 TransitionLayer; 4 touch controller; 5 screen-reader live regions.

Scaling: $scale = \text{floor}(\min(\text{availW} * \text{dpr} / 240, \text{availH} * \text{dpr} / 160))$; CSS unit --u = $scale / \text{dpr}$ CSS px; DOM sizes use $\text{calc}(\text{var}(--u) * n)$, not transform: scale. image-rendering: pixelated everywhere. On a 360 px phone at DPR 3: scale 4 = 320 CSS px wide.

Animation	Owner
Walk and idle	Phaser (driven by sim state)
Dialogue reveal, cursor, menu slide/fade	React/CSS, honoring reduced motion
Battle send-out slide	CSS
VS, battle wipe, switch, door fade	TransitionLayer
All waits and cutscene timing	Clock (injectable), never setTimeout

Synchronization: orchestrator issues a command and awaits its ack; every await has a timeout and fallback.

13. Input Architecture

- **Logical actions:** UP DOWN LEFT RIGHT A B X Y with press/release edges and held state.
- **Sources:** keyboard adapter; touch adapter (React controller calls the same entry point); gamepad later with no consumer changes.
- **Context stack:** WORLD, DIALOGUE, MENU, BATTLE, MODAL. Only the top context receives actions. X/Y consumed only by WORLD and MENU (D4).
- **Consumers:** WorldSim polls held directions at step boundaries; React UI receives edge events and translates to intents.
- **Hygiene:** preventDefault on arrows/Backspace/Enter only when game focus is active; ignore event.repeat for A/B.
- **Single activation path:** pointer, screen-reader click, and keyboard all dispatch the same intent; per-event ID guard prevents double fire.
- **Touch:** ~50% opacity at rest, more when pressed, large hit targets; keyboard stays active.

14. Routing + SEO Architecture

Route	Content	Rendering
/	Game shell, title, skip link, focusable nav to routes, noscript content	SSG shell; game client-only (dynamic import, ssr: false)
/projects	Six Party projects and summaries	SSG
/projects/[slug]	Full project page incl. all three audience trees as expandable Q&A	SSG via generateStaticParams
/skills	Pokédex content as normal list	SSG
/experience	Full entries	SSG
/about	Identity and profile	SSG
/resume	HTML resume + PDF download	SSG

- **Metadata:** generateMetadata per route, canonical URLs, sitemap.ts, robots.ts, static OG images, JSON-LD Person on /about.
- **Game vs routes:** the game never routes for in-game screens; both render the same view-models. Menu EXIT goes to /about (D5).
- **Browser Back (history sentinel):** on entering a modal flow push one sentinel entry; popstate dispatches B and re-pushes the sentinel while modal; remove it quietly on close. Fallback: no interception.
- **LINK and user activation:** reducer is synchronous so the OPEN_LINK effect runs in the same call stack as the key/click (valid user gesture). Never await between input and window.open.
- **Resume USE (D5):** opens PDF in new tab; /resume is the HTML view.
- **Static-export-ready:** avoid server-only features.

15. Asset + Audio Architecture

assets-src/ (Tiled, Aseprite, audio masters - not shipped)
public/game/ (shipped, content-hashed, with assets.manifest.json)
maps/ atlases/ portraits/ projects/<slug>/ ui/ audio/ fonts/

- JSON-hash atlases with Aseprite frame tags; lossless PNG optimization (no palette-changing quantization).
- Project art replaceable by project-<slug> without touching content IDs.
- Self-hosted woff2 subset for DOM pixel font plus readable fallback; Phaser uses no text.
- Build step hashes filenames, writes assets.manifest.json; immutable caching.

Loading tiers

Tier	Contents	When
T0	HTML/CSS, title art, pixel font	first paint
T1	Town map, tileset, player/NPC atlas, UI cursors/windows, overworld music	preload during title; narration hides remaining wait
T2	VS portraits, battle UI, battle music, first project's logo and content	idle after overworld starts
T3	Interiors, other project logos/content, collectible content	lazy, on demand or idle

Audio

AudioService built on Howler (not Phaser's sound manager, because menus need SFX without Phaser). Unlock on PRESS START. SFX as one audio sprite; music ogg + mp3 fallback, looping. Pause on visibilitychange. Persisted mute. Failed decode degrades to silence, no retry loop.

16. Persistence Architecture

Key	Storage	Content
uw.settings.v1	localStorage	music, sfx, text speed, animation mode, reduced-motion preference
uw.hints.v1	localStorage	seen hint IDs
uw.progress.v1	sessionStorage	introSeen, firstEncounterDone, discovered eggs/collectibles, visited node IDs

- All reads via Zod safeParse with versioned migrations; corrupt data resets that one key.
- If storage is unavailable, fall back to an in-memory adapter and continue.
- No cookies, so no consent banner. Player position is not persisted. Refresh restarts at spawn, no narration if introSeen (D6).
- Optional: sync settings across tabs via the storage event.

17. Performance Architecture

- **Code splitting:** Phaser loads only on / via dynamic import, never on portfolio routes. Content split per project and prefetched on demand.
- **Phaser:** import through a single game/phaser.ts so tree-shaking or a custom build can be tried later.
- **Loading:** tiered (Section 15); idle prefetch via requestIdleCallback.
- **Rendering:** fixed 240x160 internal canvas, nearest-neighbor, pixelArt: true, atlases, only camera room visible.
- **Mobile:** DPR-aware integer scaling, cap 60 FPS, pause when hidden, avoid per-frame allocations, light DOM UI.
- **Memory:** paused world stays resident during battle by design; free T2/T3 only if measurements justify.
- **Fonts/images:** font-display: block with matching fallback; Next image optimization for web layer.
- **Measurement:** Lighthouse CI on web routes, bundle-size budget test, manual low-end-phone check per release. Budgets to be set once measured.

18. Accessibility Architecture

- Semantic HTML for all professional content on routes; skip link as first tab stop.
- All in-game UI is real DOM; focus managed on screen push/pop and returned to the prior element on close.
- **Typewriter a11y:** visible typing text is aria-hidden; a polite live region holds the full page text immediately.
- **Overworld:** canvas is not screen-reader navigable; equivalent is X/Y menus and routes, plus live-region announcements for interactions and the encounter. Documented limitation.
- **Reduced motion:** prefers-reduced-motion plus Options; wipes become fades/cuts; nothing flashes. Typewriter follows text-speed independently.
- Readable text mode: post-MVP. No time limits; no color-only meaning; visible focus styled as the pixel cursor; browser zoom allowed.

19. Testing Architecture

Layer	Tooling	Scope
Unit	Vitest	tree utils, compile, validators, reducers, sim, LOS, storage, input router
Content validators	Node script + Vitest	V1-V14 against real content, every PR
State machine	Vitest + fast-check	exhaustive table, properties, switch tables
Sim replay	Vitest	recorded inputs give identical state hashes
Component	Testing Library	menus, dialogue, battle view from fixtures
Integration	Vitest + jsdom	orchestrator with fake Phaser view and fake clock
E2E	Playwright	golden path, keyboard-only, touch, reduced motion
Visual regression	Playwright screenshots	fixed-tick scenes, Chromium baselines
Accessibility	axe + manual	routes and in-game DOM screens

State machine tests: (1) for every (view, event) pair assert an explicit transition or explicit no-op; (2) property tests over random event sequences - depth ≤ 3 , focus exists in tree, BACK never leaves tree, PARTY

round trips restore a valid view, switch always lands on a valid node or root, never an undefined view; (3) switch-resolution tables including missing deep branches.

E2E hooks (NEXT_PUBLIC_E2E only, forbidden in production builds): teleport, fast clock, seed. Golden path = Title, narration, move, LOS, dialogue, audience, VS, battle, one branch, EXIT, exact position.

20. Error Handling / Resilience

Failure	Detection	Behavior
Bad content	Build validators; runtime Result	Build fails; at runtime hide project or return to parent.
Missing tree at runtime	resolveTree error	Notice + return to last valid view.
Missing/failed asset	Loader error	Placeholder; log once. Missing map: error screen with route links.
Corrupted localStorage	safeParse failure	Reset that key, continue.
Missing/invalid link	V8 + runtime guard	Button disabled with neutral notice.
Audio failure	Howler error	Continue silently.
Phaser/WebGL failure	Init error / context loss	Fallback page with route links; one restore attempt.
Unsupported browser	Feature detect	Same fallback page.
Refresh in battle	n/a	Restarts from spawn (D6).
Browser Back / route change	popstate, router events	Sentinel mapping; unmount aborts everything.
Incomplete project config	Validators	Build fails; runtime hides from Party.
Stuck transition/cutscene	Ack timeouts	Fall through to next state; log.

An error boundary wraps the game shell so a React error never takes down the routes.

21. Security / Browser Concerns

- **External links:** window.open(url, '_blank', 'noopener,noreferrer') or a rel="noopener noreferrer" anchor. Validate at build and before opening. Allow https: and mailto: only.
- **XSS:** content is plain strings rendered as text; no dangerouslySetInnerHTML. If MDX is added later, sanitize with rehype-sanitize.
- **CSP:** default-src 'self'; no third-party scripts; nonce for Next inline scripts; verify in CI that Phaser works without unsafe-eval.
- **Headers:** Referrer-Policy, X-Content-Type-Options, frame-ancestors, Permissions-Policy.
- **Local persistence:** no secrets; treat stored data as untrusted and validate on read.
- **Third-party assets:** self-host fonts, audio, libraries; keep a LICENSES record (Asset Spec Section 11).
- Email: render mailto at runtime (mild anti-scraping). Resume PDF: strip metadata.

22. Deployment Architecture

- **Hosting:** static-capable CDN. Vercel is the natural fit; static-export compatibility keeps Cloudflare Pages etc. a cheap swap. No server runtime or database.
- **Pipeline:** pnpm, pinned Node. Lint + typecheck, dependency-boundary check, content validators, unit + state tests, map and asset compile, build, Playwright on preview, Lighthouse + axe, deploy.
- **Environments:** preview per PR; production on merge to main. Env vars only NEXT_PUBLIC_SITE_URL and NEXT_PUBLIC_E2E (must be off in production, enforced in CI). No secrets.
- **Caching:** hashed game assets Cache-Control: public, max-age=31536000, immutable; HTML revalidates; resume PDF short cache.
- **Open:** analytics (none assumed).

23. Target Repository Structure

```

/
+- src/
| +- app/      Next routes: layout, page (game), projects/, skills/, experience/,
| |            about/, resume/, sitemap.ts, robots.ts
| +- domain/   types, brands, Zod schemas, limits, tree utils
| +- content/  audiences.ts party.ts project-types.ts skills.ts experience.ts
| |            trainer-card.ts bag.ts hints.ts ui-strings.ts
| |            projects/<slug>/{project.ts, recruiter.ts, engineer.ts, friend.ts}
| |            world/{npcs.ts, signs.ts, objects.ts, eggs.ts, encounter.ts}
| +- core/     battle/ encounter/ world/ flow/ input/ dialogue/ clock/
| +- runtime/  orchestrator, bridge, stores, storage adapters, audio service,
| |            asset loader, content provider, history sentinel
| +- game/     phaser boot, scenes, entity views, tilemap adapter
| +- ui/       game/ (viewport, window, menus, battle, vs, hints, transitions, controller)
| |            web/ (page components for routes)
| +- styles/   test-utils/
+- assets-src/ tiled/ aseprite/ audio/
+- public/     game/ (hashed) resume/ og/
+- scripts/    validate-content compile-maps pack-assets check-links
+- tests/      e2e/ visual/
+- docs/       adr/ architecture.md content-authoring-guide.md

```

24. Architecture Decision Records

A D R	Decision	Why	Alternatives	Trade-offs	Status
01	Keep Phaser; load only on /; render world + backdrop only.	Sprites, tilemaps, animation; isolation protects web layer.	PixiJS, custom Canvas2D, DOM tiles.	Heavy for a 3-NPC town.	Adopted
02	React renders all in-game UI as DOM over the canvas.	Accessibility, text layout, focus, testability.	All UI in Phaser; per-screen hybrid.	DOM pixel styling needs care; cross-renderer sequencing needs acks.	Optional refinement of brief
03	Phaser-free simulation core in core/world.	Deterministic, browser-free tests, replaceable renderer.	Phaser arcade physics.	Small extra glue.	Strongly recommended
04	TransitionLayer is top DOM layer.	Phaser wipe cannot cover React overlays.	Phaser-only transitions.	Extra canvas/CSS mask.	Required
05	Pure reducers for battle/encounter/flow; tiny vanilla Zustand stores for settings/progress/hints/UI.	Determinism, serializability, usable from Phaser.	XState, Redux, single global store.	No statechart visualizer; exhaustive-table test substitutes.	Adopted
06	Typed TS content + Zod + CI validators.	Type safety, nested authoring, zero infrastructure.	JSON/YAML, MDX, CMS, database.	Authors edit code files; MDX can be added later.	Adopted
07	Key-path switch resolution with nearest-valid-parent fallback.	"Same depth" ambiguous otherwise.	Depth-only; reset to root.	Authors align some keys.	Pending D2
08	Session + local storage; no cookies; no backend.	Small, per-visitor, non-sensitive.	Cookies, IndexedDB, server.	Session data dies with the tab.	Adopted
09	History sentinel maps browser Back to B in modal flows.	Avoid leaving mid-battle.	No interception.	Fiddly; documented fallback.	Needs E2E
10	Device-pixel integer scaling with --u units.	Integer rule + usable phone size.	CSS-pixel integer; fractional scale.	Letterboxing.	Pending D7
11	Static-export-compatible Next.js.	No backend needed; portable hosting.	ISR/server components.	Forgo unneeded server features.	Adopted

25. Phased Implementation Plan

Milestones: **M1** vertical slice (end of Phase 6). **M2** content-complete feature set (Phase 10). **M3** release candidate (Phase 12).

Phase	Name	Depends on	Key deliverable	Definition of Done (short)
0	Foundations + decisions lock	none	Repo skeleton, boundaries enforced, D1-D10 closed	CI green; boundary check fails on deliberate violation
1	Domain + content contracts	P0	Types, Zod, compileTree, V1-V13, seed content	Validators pass seed, fail each broken fixture
2	Battle core (headless)	P1	Complete reducer + resolveSwitch	Invariants hold under random sequences
3	Runtime spine + UI kit	P0 (overlaps P2)	Clock, InputRouter, stores, viewport, Window/Dialogue/CommandGrid	Kit crisp at several sizes and DPRs
4	Battle UI slice (no world)	P2, P3	Playable interview on a plain page	Full path, mid-depth switch, LINK, EXIT by keyboard and touch
5	World core + Phaser shell	P3 (overlaps P4)	WorldSim, MapData, GameBridge, Phaser boot	Walk, collide, talk, enter/leave interior, camera snaps
6	Encounter + flow (M1)	P4, P5	Title to exact-return slice	Golden-path E2E passes
7	In-game portfolio menus	P1, P3	Player Menu, Bag, Pokédex, Experience, Trainer Card, Projects panel, Options	Every PRD Section 4 direct path works
8	Web layer + SEO	P1 (parallel)	Six routes with real HTML	Every essential fact reachable without the game
9	Full world build-out	P6 + art	Real town, 3 interiors, NPCs, eggs	World Section 10 criteria pass
10	Content completion	P1 + your material	All 18 trees and all copy	No stubs; no unsourced claims
11	Art, audio, polish	P6 + art/audio	Final assets, reduced-motion variants	Asset Spec checklist complete
12	Hardening + release (M3)	all	RC	PRD Section 11 and Functional Section 12 pass

Full per-phase work items, contracts, tests, risks, and "do not build yet" lists are in Part II.

26. Parallel Work Plan

Track	Work	Starts	Notes
A: Engine	P2, P3, P4, P5, P6	after P1	P2/P3 overlap; P4/P5 overlap
B: Content	P10 authoring	after P1	Long pole; start early
C: Art + audio	sprites, tiles, portraits, logos, music	after P0	Placeholders unblock A until P9/P11
D: Web layer	P8	after P1	Independent of the game

Join points: P6 needs A; P7 needs A + content shape; P9 needs A + C; P12 needs everything.

27. Technical Risks + Mitigations

Risk	Mitigation
Ambiguous leaf / same-depth semantics cause rework	Close D1-D3 in Phase 0; depth is one constant.
Phaser bundle weight hurts load	Route-level split, tiered assets, isolation layer.
Crisp DOM pixel text at arbitrary DPR	Device-pixel --u units; prototype in Phase 3.
Two renderers drift out of sync	Command/ack with timeouts, single orchestrator, fake-view integration tests.
History manipulation breaks Back	Sentinel with E2E and documented fallback.
Content is the long pole	Stub status, incremental authoring, CI gate on release.
LOS trigger feels unreliable or unfair	Placement validator + playtest; tunable geometry in data.
Autoplay / audio differences	Unlock on PRESS START; silent degradation.
Accessibility bolted on late	DOM UI from Phase 3; axe in CI from Phase 3.
32x32 sprite scale looks wrong on 240x160	Early art check in Phase 5 with placeholders.

Risk	Mitigation
Invented or unsupported content claims	V12 lint, human review, no fabricated metrics.

28. Pre-Implementation Checklist

Decisions to close: D1-D10, plus confirm C17 (opening world message) and C20 (interior exit tile).

Inputs only you can provide:

- Final six projects, their order, and a real primary LINK for each.
- Real facts for every answer, so all 18 trees can be authored from source material.
- Resume file, experience entries, Trainer Card fields, Bag items, certificate details.
- Art direction and a pixel font choice with its license.
- A position on analytics (none assumed).

Setup to confirm: hosting choice, domain, Node and npnm versions, who owns art and audio.

PART II - CODING-AGENT INSTRUCTIONS

A. How to run the agents

1. Start every agent session by pasting the **Master Brief (Section B)** plus attaching this PDF and the eight design documents.
2. Then paste **one** Phase Prompt (Section C). Never give an agent two phases at once.
3. Require the **Phase Report (Section D)** at the end. Review it against the phase Definition of Done before releasing the next phase.
4. Phases P2+P3, P4+P5, and tracks B/C/D may run in parallel with separate agents and separate branches. Do not parallelize anything else.
5. The blueprint assumes nothing about your existing repository. Phase 0 therefore begins with a read-only **repo audit** and a written migration plan that you approve before any file is moved or deleted.
6. If an agent reports a conflict with the design documents, it stops and asks. It does not choose.

B. Master Brief (paste at the start of EVERY agent session)

ROLE

You are a senior engineer on a small team building a Pokemon-inspired 2D RPG developer portfolio. You implement exactly one phase at a time from the attached Engineering Blueprint. You are not the product designer and not the architect.

SOURCES OF TRUTH (in precedence order)

1. The Engineering Blueprint PDF (Part I architecture, Part II instructions).
 2. PRD v1.0, Functional Interaction/State v1.0, World/Level v1.0, First-Time Experience v1.0, Content/Data v1.0, Asset Spec v1.0. UI/UX v0.1 for visual tokens only.
 3. The existing repository (you must audit it; never assume its contents).
- If two sources conflict, STOP and report. Do not silently pick one.

STACK

Next.js (App Router), React, TypeScript (strict), Phaser, Zod, Zustand (vanilla), Howler, Vitest, fast-check, Testing Library, Playwright, axe, pnpm.

LOCKED DECISIONS (edit here if the owner changes any)

- D1 depth: root is a summary container; answers at depths 1-3; MAX_ANSWER_DEPTH = 3
- D2 party switch: resolve by shared topicKey path; fall back to nearest valid ancestor; land on the topic list at that depth; never auto-play an answer
- D3 after a leaf answer: automatically return to the parent topic list (DETAILS guard kept)
- D4 X and Y are ignored during encounter, VS, battle and transitions
- D5 Player Menu EXIT goes to /about; Bag "Resume" USE opens the PDF in a new tab
- D6 progress flags in sessionStorage; settings/hints in localStorage; refresh restarts at spawn and skips narration if introSeen
- D7 device-pixel integer scaling; landscape-first mobile
- D8 challenger LOS: straight line in facing direction, 1-3 tiles, blocked by solids
- D9 interiors exit via B and an exit tile
- D10 Player Menu > Projects opens a read-only panel (LINK + open full page), not a battle

NON-NEGOTIABLE RULES

1. Do not redesign the product. No ordinary project-card portfolio. No HP, damage, XP, win/lose, or progression. Questions are DETAILS topics; the UI command stays DETAILS.
2. Layer boundaries (enforced by dependency-cruiser in CI):
domain <- content, core; core imports only domain; game never imports React/content; ui never imports Phaser; runtime never imports Phaser objects or React components.
3. No engine or UI code may name a project (e.g. "Acko Clinic") or an audience ID ("recruiter" | "engineer" | "friend"). Those live only in content/ and tests.
Acko Clinic is default first project through PartyConfig.defaultFirst, nothing else.
4. Question trees are pure data. The battle reducer is a pure function:
(context, event, deps) -> { context, effects }. No timers, DOM, Phaser, or I/O inside.
5. No setTimeout/setInterval/Date.now in core/ or cutscenes. Use the injected Clock.
6. One owner per state variable. No global store. No store writes another store owns.
7. Phaser receives IDs, coordinates and animation keys only - never prose or content.
8. All in-game UI is real DOM (React). Transitions render in the top-most layer.
9. Content is plain strings rendered as text. No dangerouslySetInnerHTML. External links open with noopener,noreferrer and only https://mailto:.
10. LINK must call window.open synchronously in the input event call stack. No await between.
11. NEVER invent portfolio facts, metrics, links, project internals, or answers. Use clearly marked placeholders with status: 'stub'. V12 lint forbids unsourced numbers.
12. Do not add a backend, database, CMS, cookies, analytics, or new dependencies without an ADR-style note and owner approval.
13. Do not build anything listed under "Do NOT build yet" for the current phase.
14. Do not copy Nintendo/Game Freak assets, names, screenshots or audio. Placeholders only.

WORKING PROTOCOL

1. Read the phase prompt, the referenced blueprint sections, and the relevant design docs.
2. Audit the repo (read-only) for what already exists for this phase. Write a short PLAN: files to create/change, contracts, tests. If the plan would touch more than ~25 files or contradicts a locked decision, stop and ask.
3. Work in small commits on a branch named phase-<n>-<slug>. One logical change per commit.
4. Write tests with the code, not after. Run lint, typecheck, boundary check, and tests before declaring anything done.
5. Keep a running STATUS note while working (what is done / in progress / blocked).
6. Finish with the Phase Report. Do not start the next phase.

STOP AND ASK WHEN

- a document conflict or ambiguity is not already resolved in the Blueprint
- a locked decision seems wrong or blocks the work
- content facts are needed that you were not given
- you need a new dependency or a boundary exception
- a test cannot be made deterministic

Never guess. A question is cheaper than rework.

DEFINITION OF DONE (every phase)

Typecheck, lint, boundary check, content validators and all tests pass; the phase's own DoD checklist is satisfied; no TODO without an owner and issue note; docs updated; Phase Report delivered.

C. Phase prompts (paste ONE at a time, after the Master Brief)

Phase 0 - Foundations and decisions lock

PHASE 0: Foundations and decisions lock

OBJECTIVE A repo skeleton with enforced layer boundaries and all decisions recorded.

READ Blueprint Sections 0, 2, 3, 22, 23, 24; Asset Spec Section 11.

STEP 0 (read-only, then STOP for approval)

Audit the existing repository. Output: (a) inventory of what exists, (b) mapping of each existing module to the target structure (keep / move / rewrite / delete), (c) risks.
Do not modify anything until the owner approves this mapping.

STEP 1 (after approval)

1. Next.js App Router + TypeScript strict (noUncheckedIndexedAccess on), pnpm, pinned Node.
2. ESLint + Prettier; dependency-cruiser config encoding the layer table in Section 3; an ESLint rule/script that fails if the literals "recruiter", "engineer", "friend" or any project name appear outside src/content and tests.
3. Vitest configured; Playwright installed but with one smoke test only.
4. CI pipeline: lint, typecheck, boundaries, unit tests, build. Preview deploy hook.
5. Create the empty folder structure from Section 23 with README stubs.
6. docs/adr/: seed ADR-01..11 from Section 24; record decisions D1-D10 as given.
7. Record the chosen pixel font and its license in docs/LICENSES.md (ask the owner; if unset, use a clearly-marked open-license placeholder).

TESTS CI smoke; a deliberate boundary violation in a scratch branch must fail CI.

DoD CI green; boundary and literal-name checks enforced; ADRs committed; audit approved.

RISKS Existing code fighting the new structure; decisions still open.

DO NOT BUILD YET Any game, UI, or content code.

Phase 1 - Domain and content contracts

PHASE 1: Domain and content contracts

OBJECTIVE Types, schemas, tree compiler and validators so that trees are data.

READ Blueprint Sections 7, 8, 9; Content/Data Spec Sections 3-7, 12; PRD Section 8.

WORK

1. src/domain: branded IDs, AnswerBlock, TopicNode, QuestionTree, CompiledTree/Node, ProjectDef, ProjectTypeDef, LinkDef, AssetRef, AudienceDef, PartyConfig, ReactionPool, Result<T,E>, limits.ts (MAX_ANSWER_DEPTH=3, menu size limits, page length limit).
2. Zod schemas mirroring every authored type; TypeScript types inferred from or checked against them.
3. compileTree(): assigns depth/parent/childIds, returns Result with typed TreeError[].
4. getChildren(), path utilities (focus path as topicKey list), resolveTree(project, audience).
5. scripts/validate-content implementing V1-V13 (V14 comes in Phase 2). Exit non-zero on failure with file, tree and node in the message. V11 (no stubs) toggled by env flag.
6. content/: audiences.ts (3 entries as data), project-types.ts, party.ts (six seed projects, defaultFirst = Acko Clinic), project shells for all six with placeholders. Author ONE project's three trees as clearly-marked stub content (status: 'stub'). Do not invent real facts; use "TODO(owner)" placeholder text.

TESTS Unit tests per validator; a fixtures folder with one deliberately broken tree per rule V1-V13; compileTree tests (depth 4 rejected, missing answer rejected, duplicate IDs).
DoD Validators pass on seed content; each broken fixture fails with the correct error.
RISKS D1/D2 changing late - keep depth and key logic in one place.
DO NOT BUILD YET Reducer, UI, real answers for all 18 trees, any Phaser code.

Phase 2 - Battle core (headless)

PHASE 2: Battle core (headless)

OBJECTIVE The complete deterministic battle reducer with switch resolution.

READ Blueprint Section 6 (entire), Section 7; Functional Spec Sections 7-10; PRD Section 7.

WORK

1. core/battle: BattleContext, BattleEvent, BattleEffect, battleReduce(ctx, ev, deps). deps supplies tree lookup, party order, reaction pools, cooldown policy - no content imports inside core.
2. Implement every row of the Section 6 transition table, including guards, locked-state event dropping, ABORT, and the leaf DETAILS notice guard.
3. resolveSwitch() per Section 6 "Switch resolution".
4. Reaction policy: deterministic round-robin per context, cooldown counter, bypass for project-entry and project-switch.
5. Selectors/view-models: current view description, visible topics, cursor target, available commands, page text. Pure functions.
6. Effects: OPEN_LINK, PLAY_SFX, START_TRANSITION, SHOW_REACTION, BATTLE_ENDED.

TESTS

- Exhaustive (view x event) table: each pair is an explicit transition or explicit no-op.
- fast-check properties: depth ≤ 3 ; focus always exists in the tree; BACK never leaves the tree; PARTY round trip restores a valid view; resolveSwitch always lands on a valid node or root; reducer never reaches an undefined view (this is validator V14).
- resolveSwitch tables incl. missing deep branches, identical-project switch, empty path.

DoD All invariants hold over $\geq 10,000$ random sequences per property in CI.

RISKS D3 semantics at leaves; keep the guard isolated.

DO NOT BUILD YET UI, audio, transitions rendering, Phaser, persistence.

Phase 3 - Runtime spine and UI kit

PHASE 3: Runtime spine and UI kit

OBJECTIVE Everything the shells need, with no Phaser.

READ Blueprint Sections 5, 11, 12, 13, 16, 18; UI/UX Spec Sections 3-5, 17-18, 20-21, 24.

WORK

1. core/clock: Clock interface, real + fake implementations (wait, now, schedule).
2. core/input: GameAction, press/release edges, held state, context stack (WORLD, DIALOGUE, MENU, BATTLE, MODAL), per-context consumed-action declarations (X/Y only WORLD and MENU), repeat filtering, double-activation guard.
3. Keyboard adapter (Arrows, Enter, Backspace, X, Y; preventDefault only when game focused) and touch adapter entry point.
4. runtime/: settingsStore, progressStore, hintStore, uiStore (vanilla Zustand); storage adapters with Zod safeParse, versioned migration, corrupt-key reset, in-memory fallback. Keys: uw.settings.v1, uw.hints.v1, uw.progress.v1.
5. ui/game kit: GameViewport (device-pixel integer scale, --u unit, pixelated), Window, Cursor, DialogueBox (typewriter via Clock, Enter reveals, ally live region with full text, typed text aria-hidden), CommandGrid (2x2), MenuList, TransitionLayer skeleton, TouchController (visual + adapter).
6. DialogueService: request queue returning Promises.
7. Dev-only kit gallery page to inspect components at several sizes and DPRs.

TESTS Router unit tests, storage corruption tests, scaling math tests, axe on kit, component tests for typewriter/reveal/pagination with fake Clock.

DoD Kit renders crisp at 1x/2x/3x DPR and several viewport sizes; keyboard and touch both drive the same intents; axe clean.

RISKS Blurry DOM text at fractional CSS pixels - verify early.

DO NOT BUILD YET Menus, battle screens, Phaser, audio, real content.

Phase 4 - Battle UI vertical slice (no world)

PHASE 4: Battle UI vertical slice

OBJECTIVE A fully playable interview on a plain dev page, against stub content.

READ Blueprint Sections 6, 7, 11; PRD Section 7; Functional Sections 7-9; UI/UX Sections 11-13.

WORK

1. BattleScreen as a pure function of BattleContext view-models: project logo panel (placeholder art), status panel (name + Type as text), topic menu, command grid, dialogue box. Layout per Blueprint C2: visitor lower-left, Uttkarsh + project upper-right.
2. Answer paging with typewriter; Enter reveals full page then advances.
3. PartyScreen (2x3 grid, six projects from PartyConfig). Switch uses the short effect.
4. LINK effect wired to window.open synchronously (noopener,noreferrer). Invalid link -> disabled state + neutral notice.

5. Challenger reactions in the normal dialogue box (portrait shown only with reaction text).
 6. Send-out line "Uttkarsh sent out <project>!" (wording from ui-strings content).
 7. VsScreen (DOM/CSS, ~1.5 s, audience-aware title and announcement from AudienceDef).
 8. Transition specs: battle-wipe, switch-short, exit-short (placeholder visuals) with reduced-motion variants.
 9. Dev page with an audience picker and a project picker to exercise everything.

TESTS Component tests per view; E2E (Playwright, keyboard-only): full path to depth 3, party switch at depth 2, LINK, EXIT; repeat a question; touch-only pass.

DoD Every state in the Section 6 table is reachable and correct by keyboard and touch.

RISKS Nested-level presentation (topic list vs 2x2 grid) is under-specified - keep the reducer independent and document the presentation choice for owner review.

DO NOT BUILD YET Overworld, menus, real answers, final art, audio.

Phase 5 - World core and Phaser shell

PHASE 5: World core and Phaser shell

OBJECTIVE A playable test room: walk, collide, talk, enter and leave an interior.

READ Blueprint Sections 4, 10, 12; World/Level Spec Sections 2-9; Asset Spec Section 2.

WORK

1. scripts/compile-maps: Tiled JSON -> normalized MapData (tiles, collision flags, objects, rooms); validate (spawn exists, doors paired, no blocked spawn).
2. core/world: fixed-step sim, tile movement, facing, collision (SOLID, WATER), NPC routes (loop/pingpong, waits, tile reservation, blocked-by-player wait), interactions (tile in front), doors + map stack, rooms/camera snap, DialogueVariant predicates.
3. core/world/los: LOS per D8 with unit tests; active only while firstEncounterDone false.
4. runtime GameBridge: typed commands/events, Promise acks with timeouts.
5. game/: Phaser boot (pixelArt, 240x160, no anti-aliasing), tilemap adapter, sprite views driven by sim snapshots, pause/resume, backdrop lock. Loaded only via dynamic import.
6. Placeholder art only (flat tiles, colored squares). One test map + one interior.
7. Early art check: render a 32x32 placeholder sprite on the 15x10 screen; report scale feel to the owner (Blueprint C15).

TESTS Sim unit + replay-hash tests, LOS table tests, NPC deadlock test, map compiler tests, visual baseline of the test room at fixed ticks.

DoD Walk, collide, talk to NPC, enter/leave interior, camera snaps, pause/resume leaves exact state. Phaser is absent from non-game route bundles.

RISKS Step duration feel needs playtest; expose as a tunable constant.

DO NOT BUILD YET Encounter, real town, real art, menus, audio.

Phase 6 - Encounter and flow (M1 vertical slice)

PHASE 6: Encounter and flow - M1

OBJECTIVE Title -> narration -> overworld -> LOS -> dialogue -> audience -> VS -> battle -> one branch -> EXIT -> exact return.

READ Blueprint Sections 4, 10 (Encounter), 14; First-Time Experience (all); Functional Sections 3-6, 10.

WORK

1. core/flow: FlowMachine (TITLE, INTRO, OVERWORLD, INTERIOR, ENCOUNTER, VS, BATTLE, ...) with an explicit transition table.
2. Title screen (minimal logo, PRESS START; unlocks audio), narration with locked copy from content, brief town reveal via room-snap script, then control.
3. core/encounter: step runner (wait, face, exclaim, move, say, choice, vs, startBattle). First-encounter script with locked dialogue; repeat script shorter with audience greeting. Anticipation 200-400 ms via Clock. Challenger faces player; approach stops one tile away; NO mirroring of player movement.
4. runtime Orchestrator: owns machines, runs effects, awaits bridge acks with timeouts, stores the encounter anchor (map, tile, facing), pauses the world, mounts battle, and on BATTLE_ENDED restores the exact anchor.
5. HintService: first-move and first-interact hints only.
6. History sentinel (Blueprint Section 14) with documented fallback switch.
7. Set progressStore.firstEncounterDone on first completed encounter.

TESTS Integration with fake Clock and fake Phaser view; golden-path Playwright E2E using E2E hooks (teleport, fast clock); refresh restarts cleanly; Back button at root exits battle; repeat encounter requires Enter and offers audience selection again.

DoD Full slice runs; EXIT returns to exact tile and facing; first encounter cannot be skipped once LOS met; repeat encounter works; no stuck state after interrupts.

RISKS Ack timeouts and cutscene interruption; test aborts mid-sequence.

DO NOT BUILD YET Player Menu/Bag/etc., final map, full content, final art/audio.

Phase 7 - In-game portfolio menus

PHASE 7: In-game portfolio menus

OBJECTIVE Every essential fact reachable directly from X/Y.

READ Blueprint Section 11; PRD Sections 4-5; Functional Section 5; UI/UX Sections 6-9, 15-17.

WORK

1. MenuDef data and ScreenStack; Player Menu: POKEDEX, PROJECTS, EXPERIENCE, BAG, UTTKARSH, OPTIONS, EXIT (Menu EXIT -> /about).
2. Bag (Documents, Profiles, Contact, optional Extras): description + USE. Resume USE opens PDF in new tab. Every link goes through the shared safe-open utility.
3. Pokedex list/detail (category grouping, optional search, Type text + icon, no scores).
4. Experience list/detail. Trainer Card (single screen). Projects panel (D10).
5. Options: Music, SFX, Text Speed, Animation, Reduced Motion, Controls view, Reset Tutorial.
6. X/Y available immediately in overworld and disabled per D4. Remaining hints.
7. All screens read selectors shared with the web routes; placeholder content allowed.

TESTS Component + axe per screen; E2E for each direct path in PRD Section 4

(Resume: Y > Bag > Resume; Projects; Skills; Experience; Trainer Card; GitHub/LinkedIn/Contact).

DoD Every direct path works by keyboard and touch; focus returns correctly on B.

RISKS Pokedex density at 240x160.

DO NOT BUILD YET Final art, real content beyond what exists, new menu items.

Phase 8 - Web layer and SEO (parallel from Phase 1)

PHASE 8: Web layer and SEO

OBJECTIVE All six routes render real semantic HTML from the same content.

READ Blueprint Sections 14, 18, 21, 22; Content/Data Section 11; PRD Section 10.

WORK

1. Routes: /, /projects, /projects/[slug] (generateStaticParams), /skills, /experience, /about, /resume. / ships the game shell with skip link, focusable route nav, noscript.
2. /projects/[slug] renders overview, impact, technologies, links, and all three audience trees as expandable Q&A (details/summary, no JS required).
3. generateMetadata per route, canonical URLs, sitemap.ts, robots.ts, OG images, JSON-LD Person on /about. The game title screen itself has no professional subtitle.
4. /resume HTML view + PDF download; PDF metadata stripped.
5. Security headers and CSP (Section 21). Static-export compatibility check.

TESTS Route render tests, axe per route, Lighthouse CI, link checker script.

DoD Every essential professional fact is reachable with JS disabled.

RISKS Content not ready - use stub banners that CI blocks in release builds.

DO NOT BUILD YET Game integration beyond skip link and Menu EXIT target.

Phase 9 - Full world build-out

PHASE 9: Full world build-out

OBJECTIVE The real town.

READ World/Level Spec (all); Blueprint Section 10; Asset Spec Sections 3-4.

WORK

1. Final town map (2x2 to 3x3 screens): center spawn, 3 enterable buildings (one reads as home, no labels), pond/stream, grass patch (cosmetic), tiny side forest route, grand tree landmark, signs. 3 interiors.
2. 2 ordinary NPCs with routes, idle cues, contextual dialogue variants. 1 challenger at a fixed spot visible on a normal route; LOS lane validated.
3. Easter eggs and a few hidden collectibles (medium discovery, none on the critical path, no counter); discovery persists to progressStore.
4. Placement validator script: lane reachable from spawn, lane tiles walkable, interiors paired, nothing essential only inside buildings.

TESTS Placement validator, replay tests across the lane, E2E for discovery persistence.

DoD All World Section 10 acceptance criteria pass.

RISKS First-encounter reliability; playtest from several approach directions.

DO NOT BUILD YET New mechanics, extra buildings/NPCs, collectible counters.

Phase 10 - Content completion (parallel from Phase 1)

PHASE 10: Content completion

OBJECTIVE All 18 trees and all copy authored from real source material.

INPUT REQUIRED FROM OWNER real facts per project, links, resume, experience, trainer card.

WORK Agents may structure, lint, and format content the owner supplies. Agents must NOT write factual claims the owner did not provide. Replace each stub with authored content supplied by the owner; set status: 'authored' per tree.

RULES Level 1 topicKeys match the shared vocabulary (what/how/why/hard). Friend trees still reach all facts. Recruiter = ownership/impact; Engineer = architecture/trade-offs; Friend = motivation/stories. Audience trees must differ substantively, not by word swap.

TESTS V1-V13 with V11 strict; human review against the truth-before-flavor principle.

DoD No stubs; every claim traceable to owner-provided material.

DO NOT BUILD YET Anything engine-related.

Phase 11 - Art, audio, polish

PHASE 11: Art, audio, polish

OBJECTIVE Replace placeholders; tune feel to First-Time Experience targets.

READ Asset Spec (all); UI/UX Sections 21-23; FTE Section 11.

WORK Integrate final atlases, portraits, project logos, fonts, SFX, music via the asset manifest and tiering. AudioService (Howler), unlock on PRESS START, persisted mute.

Final transition visuals with reduced-motion variants. Tune timings (anticipation 200-400 ms, VS ~1.5 s, encounter 6-10 s excluding reading).

TESTS Visual regression re-baseline; audio failure path test; reduced-motion E2E.

DoD Asset Spec production checklist complete; licenses documented.

DO NOT BUILD YET New features.

Phase 12 - Hardening and release

PHASE 12: Hardening and release

OBJECTIVE Release candidate.

WORK Accessibility audit (keyboard, screen reader, reduced motion, zoom); performance

budgets enforced in CI; CSP verification; error-path tests for every Section 20 row; cross-browser and real-device checks; licensing review; link check; E2E flags off in prod.

TESTS Everything in Blueprint Section 19.

DoD All PRD Section 11 acceptance criteria and Functional Section 12 acceptance tests pass.

DO NOT BUILD YET Post-MVP ideas: battle deep links, readable-text mode, MDX.

D. Phase Report template (agents must end every phase with this)

PHASE REPORT - Phase <n>: <name>

1. Summary (3-5 lines)
2. Files created / changed (grouped by layer)
3. Contracts introduced or changed (types, events, effects)
4. Tests added and results (counts, property-test iterations)
5. DoD checklist (each item: PASS / FAIL + evidence)
6. Deviations from the Blueprint (must be empty or justified; list any doc conflicts found)
7. Open questions for the owner
8. Risks observed
9. Anything intentionally NOT built
10. Suggested next phase and prerequisites

E. Escalation triggers (agent must stop and ask)

Trigger	Example
Doc conflict not already resolved	New contradiction found between PRD and UI/UX
Locked decision blocks work	D3 leaf behavior makes a required test impossible
Missing facts	An answer, link, or resume detail is needed
New dependency or boundary exception	Adding XState or importing Phaser in ui/
Non-deterministic test	Flaky timing that needs a real timer
Plan exceeds ~25 files or crosses phases	Work starting to touch menus during the battle phase

F. Owner review checklist (per phase)

- Does the report show every DoD item as PASS with evidence?
- Do CI, boundary check, and the no-literal-names check pass?
- Any deviation listed? Approve or reject it explicitly.
- Did the agent invent any content, link, or metric? (Reject if yes.)
- Is anything built that the phase said not to build?
- Play the result yourself (keyboard, then touch) before releasing the next phase.

G. Pre-flight before Phase 0

1. Close D1-D10 (and confirm C17, C20); edit the Locked Decisions block if you change any.
2. Decide hosting, domain, Node/pnpm versions, pixel font.
3. Gather content inputs for Track B: final six projects, links, facts, resume.
4. Assign art and audio ownership (Track C).
5. Give the agent access to the repository plus this PDF and the eight design documents.